

Ottimizzazione della Memoria nella Programmazione Dinamica per l'Allineamento di Sequenze

Iacopo Giansanti

September 2026

Indice

1	Introduzione	3
2	Definizione del Global Alignment Problem e dei prerequisiti essenziali	3
2.1	Rappresentazione delle sequenze biologiche	3
2.2	Allineamento e modello di scoring	4
2.3	Programmazione dinamica	5
2.4	Longest Common Subsequence	6
2.5	Alignment graph	8
2.6	Global Alignment Problem	8
3	Algoritmo di Needleman-Wunsch	9
3.1	Ricorrenza, inizializzazione e traceback	9
3.2	Pseudocodice	11
3.3	Analisi della complessità	12
3.4	Implementazione	13
3.4.1	Strutture dati	14
3.4.2	Algoritmo	14
4	Algoritmo di Hirschberg	15
4.1	Ricorrenza, Middle vertex e Middle edge	16
4.2	Pseudocodice	18
4.3	Analisi della complessità	21
4.4	Implementazione	23
4.4.1	Calcolo degli score in spazio lineare	23
4.4.2	Middle vertex e middle edge	24
4.4.3	Ricostruzione ricorsiva dell'allineamento	25

5	Metodologia sperimentale	26
5.1	Dataset	26
5.2	Ambiente hardware e software	27
5.3	Misura dei tempi e della memoria	27
6	Risultati e confronto	27
6.1	Tempo di esecuzione	27
6.2	Consumo di memoria	27
6.3	Scalabilità	27
6.4	Needleman-Wunsch vs Hirschberg	27

1 Introduzione

2 Definizione del Global Alignment Problem e dei prerequisiti essenziali

2.1 Rappresentazione delle sequenze biologiche

Il DNA è costituito da quattro tipi di nucleotidi, caratterizzati dalle seguenti basi azotate:

- Adenina, A;
- Citosina, C;
- Guanina, G;
- Timina, T.

I due filamenti della molecola sono complementari e antiparalleli, con appaiamenti

$$A \longleftrightarrow T, \quad C \longleftrightarrow G$$

Dal punto di vista computazionale, un singolo filamento può essere rappresentato come una stringa sull'alfabeto

$$\Sigma_{\text{DNA}} = \{\mathbf{A}, \mathbf{C}, \mathbf{G}, \mathbf{T}\}$$

Quindi una stringa di lunghezza n ha forma

$$x = x_1 x_2 \cdots x_n, \quad x_i \in \Sigma_{\text{DNA}} \quad \forall i \in \{1, \dots, n\}.$$

Una *sottostringa* è una porzione contigua di una stringa, della forma

$$x_i x_{i+1} \cdots x_j,$$

con $1 \leq i \leq j \leq n$, e n la lunghezza della stringa originale. In una stringa di lunghezza n , una sottostringa di lunghezza k esiste per $k \leq n$.

Un *k-mero* è una sottostringa di lunghezza $k \geq 1$.

Data una stringa, una *sottosequenza* è invece una stringa ottenuta eliminando 0 o più elementi dalla stringa originale mantenendone l'ordine relativo.

Mostriamo un esempio: sia s una stringa che rappresenta una sequenza genomica,

$$s := \mathbf{ACCT},$$

allora $\mathbf{AC}$ può essere sia una sottostringa che una sottosequenza, mentre $\mathbf{AT}$ è definibile soltanto come sottosequenza.

Due sequenze genomiche omologhe possono differire tra loro a causa di mutazioni. Consideriamo tre eventi possibili associati a tali mutazioni:

$$\begin{array}{ccc} \mathbf{AC} & \xrightarrow{\text{sostituzione}} & \mathbf{AA} \\ \mathbf{AC} & \xrightarrow{\text{inserimento}} & \mathbf{AAC} \\ \mathbf{AAC} & \xrightarrow{\text{cancellazione}} & \mathbf{AC} \end{array}$$

Gli ultimi due eventi verranno talvolta trattati come appartenenti a un unico tipo, dal nome *indel*.

Supponiamo, quindi, di avere due sequenze genomiche omologhe che differiscono a causa di mutazioni genetiche; come possiamo quantificare e individuare la somiglianza tra le due sequenze?

2.2 Allineamento e modello di scoring

Stiamo quindi iniziando a definire il problema di nostro interesse.

Dato che un nostro obiettivo è quello di misurare la differenza tra due sequenze genetiche, trattiamo adesso la *distanza di Hamming*, che è lo strumento più semplice per valutare la somiglianza tra due stringhe. Siano x, y due stringhe della stessa lunghezza, allora la distanza di Hamming è

$$d_H(x, y) := |\{i : x_i \neq y_i\}|,$$

ovvero il numero di posizioni in cui due stringhe differiscono. La distanza di Hamming è però applicabile soltanto a stringhe della stessa lunghezza e non modella adeguatamente inserzioni e delezioni, che possono inoltre alterare la corrispondenza tra le posizioni delle due sequenze.

Abbiamo quindi bisogno di una misura più generale. Definiamo allora l'*edit distance*, o anche distanza di Levenshtein, $d(v, w)$, come il numero minimo di operazioni elementari di inserimento, cancellazione e sostituzione necessarie per trasformare una sequenza v in una sequenza w .

L'edit distance fornisce una misura della dissimilarità tra due sequenze. Per rappresentare esplicitamente le corrispondenze tra i loro simboli introduciamo ora il concetto di **allineamento**.

Siano v e w due sequenze. Un allineamento di v e w è una matrice

$$A \in (\Sigma_{\text{DNA}} \cup \{-\})^{2 \times L},$$

con

$$\max\{|v|, |w|\} \leq L \leq |v| + |w|,$$

tale che, eliminando i simboli di gap (' - ') dalla prima riga, si ottiene v , mentre eliminandoli dalla seconda si ottiene w . Non sono ammesse colonne contenenti due gap.

Ogni colonna di un allineamento corrisponde a uno dei seguenti casi:

- match (corrispondenza);
- mismatch (sostituzione);
- inserimento;
- cancellazione;

Visualizziamo cosa è stato appena detto: siano v_{aligned} e w_{aligned} le sequenze allineate e A il loro allineamento, allora un esempio potrebbe essere:

$$\begin{array}{rcll} v_{\text{aligned}} & : & \text{A} & \text{C} & \text{T} & - & \text{G} & \text{C} & \text{A} \\ w_{\text{aligned}} & : & \text{A} & - & \text{T} & \text{C} & \text{G} & \text{C} & \text{A} \end{array} = A$$

Per valutare la similarità tra simboli introduciamo una funzione di **scoring**,

$$\delta : ((\Sigma_{\text{DNA}} \cup \{-\})^2 \setminus \{(-, -)\}) \rightarrow \mathbb{R},$$

che, dati due simboli a e b , restituisce il contributo di score associato al loro allineamento.

Una funzione di scoring (anche detta modello) per una sequenza di DNA è generalmente definita attribuendo un premio in caso di match e una penalità in caso contrario

$$\delta(a, b) = \begin{cases} \mu, & a = b, & a, b \in \Sigma_{\text{DNA}} \\ -\eta, & a \neq b, & a, b \in \Sigma_{\text{DNA}} \\ -\sigma, & \text{se esattamente uno tra } a, b \text{ è un gap} \end{cases}$$

con $\mu, \eta, \sigma > 0$. Il modello in cui mismatch e gap hanno la stessa penalità si ottiene come caso particolare ponendo $\eta = \sigma$.

Grazie alla funzione di scoring possiamo anche definire la **scoring matrix**,

δ	A	C	G	T	-
A	μ	$-\eta$	$-\eta$	$-\eta$	$-\sigma$
C	$-\eta$	μ	$-\eta$	$-\eta$	$-\sigma$
G	$-\eta$	$-\eta$	μ	$-\eta$	$-\sigma$
T	$-\eta$	$-\eta$	$-\eta$	μ	$-\sigma$
-	$-\sigma$	$-\sigma$	$-\sigma$	$-\sigma$	n.a.

Siano x, y le sequenze allineate corrispondenti rispettivamente a due stringhe v e w , quindi $x_i, y_i \in \Sigma_{\text{DNA}} \cup \{-\}$, $\forall i \in \{1, \dots, L\}$.

Sia la matrice di allineamento

$$A = \begin{pmatrix} x_1 & \cdots & x_L \\ y_1 & \cdots & y_L \end{pmatrix},$$

allora, definiamo lo score complessivo dell'allineamento come

$$\text{score}(A) = \sum_{i=1}^L \delta(x_i, y_i).$$

Il *problema dell'allineamento globale* (Global Alignment Problem) consiste nel trovare, date due sequenze, un allineamento di score massimo.

Nel modello precedente ogni gap viene penalizzato indipendentemente. Una successione di gap consecutivi può tuttavia essere interpretata come risultato di un unico evento di inserimento o cancellazione. Per questo motivo vengono spesso utilizzate penalità affini, che distinguono il costo di apertura di un gap da quello della sua estensione. Tale generalizzazione richiede stati aggiuntivi nella programmazione dinamica; nel seguito ci concentreremo tuttavia sul modello a penalità lineare utilizzato dagli algoritmi oggetto del confronto.

2.3 Programmazione dinamica

La *programmazione dinamica* è una tecnica algoritmica adatta a problemi caratterizzati da sottoproblemi sovrapposti e da una proprietà di sottostruttura ottima. Le soluzioni dei sottoproblemi vengono memorizzate, evitando di calcolarli più volte.

Nel problema di allineamento che considereremo, i sottoproblemi possono essere indicizzati mediante due indici i e j . In particolare, una ricorrenza avrà tipicamente la forma

$$P(i, j) = F(P(i-1, j), P(i, j-1), P(i-1, j-1)).$$

Siano $|v| = n$ e $|w| = m$. Indicheremo con

$$S_{i,j}$$

il valore ottimo relativo ai prefissi

$$v_1 \cdots v_i \quad \text{e} \quad w_1 \cdots w_j,$$

con $0 \leq i \leq n$ e $0 \leq j \leq m$.

Nel caso dell'allineamento di sequenze, lo stato $S_{i,j}$ dipenderà dai tre stati

$$S_{i-1,j}, \quad S_{i,j-1}, \quad S_{i-1,j-1},$$

che corrisponderanno rispettivamente a una cancellazione, un inserimento e a un match o mismatch.

Memorizzando esplicitamente tutti gli stati è necessaria una matrice di $(n+1) \times (m+1)$ elementi, e quindi uno spazio $O(nm)$.

Una volta calcolato il valore ottimo, una soluzione corrispondente può essere ricostruita memorizzando le scelte effettuate e percorrendole a ritroso mediante *traceback*.

2.4 Longest Common Subsequence

Una *sottosequenza comune* a due stringhe v e w è una stringa che è una sottosequenza sia di v sia di w .

Quindi, siano

$$v = v_1 v_2 \cdots v_n, \quad w = w_1 w_2 \cdots w_m.$$

Una stringa

$$z = z_1 z_2 \cdots z_k$$

è una sottosequenza comune a v e w se esistono indici

$$1 \leq i_1 < i_2 < \cdots < i_k \leq n$$

e

$$1 \leq j_1 < j_2 < \cdots < j_k \leq m$$

tali che

$$z_t = v_{i_t} = w_{j_t}, \quad \forall t \in \{1, \dots, k\}$$

Facciamo un esempio per chiarire:

Siano

$$v := \text{ACCT}, \quad w := \text{ACT}.$$

Allora, la stringa

$$z = \text{ACT}$$

è una sottosequenza comune.

Tra tutte le sottosequenze comuni a v e w , siamo interessati ad una di lunghezza massima, detta **Longest Common Subsequence** (LCS).

Indichiamo con $s(v, w)$ la lunghezza di una LCS tra v e w :

$$s(v, w) = \max\{|z| : z \text{ è una sottosequenza comune di } v \text{ e } w\}$$

Considerando una variante dell'edit distance in cui sono ammesse soltanto inserzioni e delezioni, trasformare v in w utilizzando una sottosequenza comune di lunghezza k richiede

$$(n - k) + (m - k)$$

operazioni. Per una LCS, con $k = s(v, w)$, si ottiene quindi tale distanza come

$$d_{\text{indel}}(v, w) = n + m - 2s(v, w)$$

Riprendiamo l'esempio precedente utilizzando le stesse sequenze v e w . La stringa

$$z = \text{ACT}$$

è una sottosequenza di entrambe le sequenze ma è inoltre una LCS, poiché $|w| = 3$ e non può quindi esistere una sottosequenza comune più lunga.

Pertanto,

$$s(v, w) = 3$$

e

$$d_{\text{indel}}(v, w) = 4 + 3 - 2 \times 3 = 1.$$

Il problema della LCS può essere ricondotto alla ricerca di un cammino di peso massimo da $(0, 0)$ a (n, m) su un digrafo aciclico definito in modo appropriato come segue.

A ogni coppia di prefissi

$$v_1 \cdots v_i, \quad w_1 \cdots w_j$$

associamo un nodo (i, j) .

Ogni nodo (i, j) può avere tre tipi di archi entranti:

- $(i-1, j) \rightarrow (i, j)$ (arco 'verticale', corrispondente all'omissione di un simbolo di v);
- $(i, j-1) \rightarrow (i, j)$ (arco 'orizzontale', corrispondente all'omissione di un simbolo di w);
- $(i-1, j-1) \rightarrow (i, j)$, soltanto quando $v_i = w_j$ (arco 'diagonale', corrispondente a un match).

Nel problema della LCS assegniamo un peso a ogni arco, $c(e)$, dove $e \in E$, con E insieme degli archi della griglia.

$$c(e) = \begin{cases} 1, & \text{arco diagonale corrispondente a un match,} \\ 0, & \text{arco verticale o orizzontale} \end{cases}$$

Il problema della LCS può quindi essere ricondotto alla ricerca di un cammino di peso massimo da $(0, 0)$ a (n, m) nel DAG associato alla griglia descritta sopra.

Sia $S_{i,j}$ la lunghezza di una LCS tra $v_1 \cdots v_i$ e $w_1 \cdots w_j$. Siano

$$S_{i,0} = 0, \quad S_{0,j} = 0.$$

Allora vale la ricorrenza:

$$S_{i,j} = \max \begin{cases} S_{i-1,j} \\ S_{i,j-1} \\ S_{i-1,j-1} + 1 \quad \text{se } v_i = w_j \end{cases}$$

I primi due termini corrispondono rispettivamente a non utilizzare v_i oppure w_j nella sottosequenza comune. Se invece $v_i = w_j$, è possibile utilizzare entrambi i simboli aggiungendo 1 alla soluzione relativa ai prefissi precedenti. La struttura della ricorrenza riprende quindi la dipendenza dai tre sottoproblemi introdotti nella sezione precedente.

Poiché gli stati sono $(n+1) \times (m+1)$, memorizzare l'intera matrice richiede spazio

$$O(nm).$$

Poiché ogni stato viene calcolato utilizzando un numero costante di operazioni, anche il tempo di calcolo è

$$O(nm).$$

Se interessa soltanto la lunghezza della LCS, è sufficiente mantenere in memoria le righe necessarie al calcolo, riducendo così lo spazio a $O(\min(n, m))$.

2.5 Alignment graph

La rappresentazione tramite grafo introdotta per la LCS può essere generalizzata al problema dell'allineamento di sequenze.

Siano v e w due sequenze, con $|v| = n$ e $|w| = m$. Definiamo l'**alignment graph** come il grafo diretto aciclico i cui nodi sono le coppie

$$(i, j), \quad 0 \leq i \leq n, \quad 0 \leq j \leq m,$$

dove il nodo (i, j) rappresenta l'allineamento dei prefissi

$$v_1 \cdots v_i, \quad w_1 \cdots w_j,$$

e i cui archi sono definiti come segue:

Direzione	Arco	Operazione nell'allineamento
verticale	$(i-1, j) \rightarrow (i, j)$	cancellazione / v_i contro gap
orizzontale	$(i, j-1) \rightarrow (i, j)$	inserimento / gap contro w_j
diagonale	$(i-1, j-1) \rightarrow (i, j)$	match o mismatch

Lo stato iniziale è $(0, 0)$ e quello finale è (n, m) .

Ogni allineamento tra v e w corrisponde a un cammino nell'alignment graph da $(0, 0)$ a (n, m) e, viceversa, ogni cammino tra questi nodi rappresenta un possibile allineamento.

A questo punto è importante mostrare un esempio per visualizzare l'alignment graph. Siano $v := \text{ACCT}$, $w := \text{ACT}$ due sequenze. Sia A il loro allineamento. Allora possiamo elencare gli archi che compongono un cammino da $(0, 0)$ a $(4, 3)$ nell'alignment graph associato ad A :

$$(0, 0) \rightarrow (1, 1) \rightarrow (2, 2) \rightarrow (3, 2) \rightarrow (4, 3).$$

Gli archi corrispondono a:

arco	colonna dell'allineamento
$(0, 0) \rightarrow (1, 1)$	(A, A)
$(1, 1) \rightarrow (2, 2)$	(C, C)
$(2, 2) \rightarrow (3, 2)$	$(C, -)$
$(3, 2) \rightarrow (4, 3)$	(T, T)

producendo quindi:

$$A = \begin{array}{cccc} & A & C & C & T \\ A & C & - & T \end{array}$$

Il peso di ogni arco è definito come segue:

$$\begin{aligned} w((i-1, j) \rightarrow (i, j)) &= \delta(v_i, -), \\ w((i, j-1) \rightarrow (i, j)) &= \delta(-, w_j), \\ w((i-1, j-1) \rightarrow (i, j)) &= \delta(v_i, w_j), \end{aligned}$$

con δ la funzione di scoring.

Il peso di un cammino coincide quindi con lo score dell'allineamento corrispondente.

2.6 Global Alignment Problem

Definiamo ora formalmente il problema dell'allineamento globale, che costituisce il problema di riferimento per gli algoritmi analizzati nel seguito.

Siano v e w due sequenze e sia δ la funzione di scoring definita precedentemente.

Dato un allineamento

$$A = \begin{pmatrix} x_1 & x_2 & \cdots & x_r \\ y_1 & y_2 & \cdots & y_r \end{pmatrix}$$

costituito da r colonne, definiamo il suo score complessivo come

$$\text{score}(A) = \sum_{k=1}^r \delta(x_k, y_k)$$

Sia $\mathcal{A}(v, w)$ l'insieme di tutti i possibili allineamenti tra v e w . Il **Global Alignment Problem** consiste nel determinare un allineamento di score massimo, ovvero

$$\text{OPT}(v, w) = \max_{A \in \mathcal{A}(v, w)} \text{score}(A)$$

Nel modello di scoring lineare considerato di seguito,

$$\text{match} = +1, \quad \text{mismatch} = -\eta, \quad \text{indel} = -\sigma,$$

con $\eta, \sigma > 0$.

Lo score di un allineamento può quindi essere espresso come

$$\text{score}(A) = \#\text{match} - \eta \#\text{mismatch} - \sigma \#\text{gap},$$

dove $\#\text{gap}$ indica il numero di colonne dell'allineamento contenenti un gap dovuto a una trasformazione indel.

Per quanto osservato nella sezione precedente, ogni allineamento corrisponde a un cammino da $(0, 0)$ a (n, m) nell'alignment graph e il peso del cammino coincide con lo score dell'allineamento. Il Global Alignment Problem può quindi essere interpretato come la ricerca di un cammino di peso massimo tra questi due nodi.

Da questa formulazione emergono tre obiettivi computazionali che guideranno l'analisi degli algoritmi successivi:

- calcolare il valore ottimo $\text{OPT}(v, w)$;
- ricostruire un allineamento che realizzi tale valore;
- ricostruire un allineamento ottimo riducendo l'utilizzo di memoria.

3 Algoritmo di Needleman-Wunsch

In questo capitolo analizzeremo l'algoritmo di Needleman-Wunsch, che è un algoritmo di programmazione dinamica per il problema dell'allineamento globale di sequenze.

3.1 Ricorrenza, inizializzazione e traceback

Sia $S_{i,j}$ il miglior score globale tra $v_1 \cdots v_i$ e $w_1 \cdots w_j$, con $0 \leq i \leq n, 0 \leq j \leq m$. Quindi, come visto nel precedente capitolo, possiamo affermare che $S_{i,j}$ coincide con il peso di un cammino massimo da $(0, 0)$ a (i, j) .

Consideriamo l'ultima colonna di un allineamento ottimo, abbiamo tre possibilità:

- **Delezione**

L'ultima colonna è

$$\begin{pmatrix} v_i \\ - \end{pmatrix}$$

quindi prima di essa è stato allineato $v_1 \cdots v_{i-1}$ con $w_1 \cdots w_j$.
Lo score in questo caso è

$$S_{i-1,j} + \delta(v_i, -)$$

- **Inserzione**

L'ultima colonna è

$$\begin{pmatrix} - \\ w_j \end{pmatrix}$$

quindi prima di essa è stato allineato $v_1 \cdots v_i$ con $w_1 \cdots w_{j-1}$.
Lo score in questo caso è

$$S_{i,j-1} + \delta(-, w_j)$$

- **Match o mismatch**

L'ultima colonna è

$$\begin{pmatrix} v_i \\ w_j \end{pmatrix}$$

quindi prima di essa è stato allineato $v_1 \cdots v_{i-1}$ con $w_1 \cdots w_{j-1}$.
Lo score in questo caso è

$$S_{i-1,j-1} + \delta(v_i, w_j)$$

Per gli stati interni dell'allineamento, con $1 \leq i \leq n$ e $1 \leq j \leq m$, possiamo formulare la ricorrenza a partire dai tre possibili predecessori. Prima però è necessario inizializzare gli stati sul bordo della matrice, che corrispondono ai casi base del problema dell'allineamento globale:

$$\begin{aligned} S_{0,0} &= 0; \\ S_{i,0} &= S_{i-1,0} + \delta(v_i, -) = -i\sigma; \\ S_{0,j} &= S_{0,j-1} + \delta(-, w_j) = -j\sigma. \end{aligned}$$

A differenza della LCS, gli stati sul bordo non hanno valore nullo: nel global alignment l'intero prefisso deve essere allineato. Allineare $v_1 \cdots v_i$ con la stringa vuota richiede quindi i gap, mentre allineare la stringa vuota con $w_1 \cdots w_j$ ne richiede j .

Possiamo, quindi, formalizzare la *ricorrenza*:

$$S_{i,j} = \max \begin{cases} S_{i-1,j} + \delta(v_i, -), \\ S_{i,j-1} + \delta(-, w_j), \\ S_{i-1,j-1} + \delta(v_i, w_j) \end{cases}$$

con $1 \leq i \leq n$ e $1 \leq j \leq m$.

Questa relazione costituisce la ricorrenza fondamentale dell'algoritmo di Needleman-Wunsch.

Il calcolo di $S_{n,m}$ soddisfa il primo degli obiettivi formulati nel capitolo precedente, fornendo

$$S_{n,m} = \text{OPT}(v, w).$$

Per ricostruire anche un allineamento che realizzi tale score è necessario mantenere l'informazione relativa alle scelte effettuate durante il calcolo. Quindi, durante il calcolo di $S_{i,j}$, possiamo memorizzare quale predecessore ha prodotto il massimo all'interno di una matrice B :

$$B_{i,j} \in \{\uparrow, \leftarrow, \nearrow\}.$$

Sul bordo della matrice il predecessore è univocamente determinato:

$$\begin{aligned} B_{i,0} &= \uparrow, & i &= 1, \dots, n, \\ B_{0,j} &= \leftarrow, & j &= 1, \dots, m. \end{aligned}$$

A ciascuna direzione memorizzata in $B_{i,j}$ corrisponde una colonna dell'allineamento secondo la seguente associazione:

$B_{i,j}$	colonna dell'allineamento
$\uparrow$	$(v_i, -)$
$\leftarrow$	$(-, w_j)$
$\swarrow$	(v_i, w_j)

Così facendo, partendo da (n, m) possiamo ricostruire l'allineamento percorrendo i predecessori fino a $(0, 0)$. Notiamo però che tramite questo processo di *traceback* le colonne vengono generate in ordine contrario, quindi al termine vanno invertite.

In caso di parità tra più direzioni esistono più cammini equivalenti, e quindi potenzialmente più allineamenti ottimi. Se l'obiettivo è ricostruire un singolo allineamento ottimo, è sufficiente adottare una regola deterministica di tie-breaking; per rappresentare tutti gli allineamenti ottimi sarebbe invece necessario conservare tutti i predecessori che realizzano il massimo.

Mostriamo un esempio. Siano $v := \text{ACCT}$ e $w := \text{ACT}$ due sequenze. Sia $\delta(a, b)$ la funzione di scoring definita in modo tale che restituisca 1 in caso di match e -1 altrimenti. Allora la matrice degli score è:

	-	A	C	T
-	0	-1	-2	-3
A	-1	1	0	-1
C	-2	0	2	1
C	-3	-1	1	1
T	-4	-2	0	2

Quindi,

$$S_{4,3} = 2$$

Il traceback sarà:

$$(4, 3) \rightarrow (3, 2) \rightarrow (2, 2) \rightarrow (1, 1) \rightarrow (0, 0),$$

ricostruendo

A	C	C	T
A	C	-	T

3.2 Pseudocodice

Possiamo adesso fornire lo pseudocodice per l'algoritmo di Needleman-Wunsch.

Listing 1: Global Alignment

```

NeedlemanWunsch(v,w,delta)
  n <- |v|
  m <- |w|

  // inizializzazione
  S[0,0] <- 0
  for i <- 1,...,n
    S[i,0] <- S[i-1,0] + delta(v[i], '-')
    B[i,0] <- up
  for j <- 1,...,m
    S[0,j] <- S[0,j-1] + delta('-', w[j])
    B[0,j] <- left

  // ricorrenza
  for i <- 1,...,n
    for j <- 1,...,m

```

```

    up      <- S[i-1,j] + delta(v[i],'-')
    left    <- S[i,j-1] + delta('-',w[j])
    diagonal <- S[i-1,j-1] + delta(v[i],w[j])
    S[i,j] <- max(up, left, diagonal)
    B[i,j] <- predecessore che ha prodotto il massimo

    // ricostruzione dell'allineamento
    alignment <- Backtrack(B,v,w,n,m)

    return (S[n,m] , alignment)

```

Adottiamo una politica deterministica di tie-breaking che assegna priorità, nell'ordine, allo spostamento diagonale, verticale e orizzontale. Il seguente è un possibile pseudocodice del traceback.

Listing 2: Ricostruzione dell'allineamento tramite traceback

```

Backtrack(B,v,w,i,j)
    alignedV <- empty
    alignedW <- empty

    while i > 0 or j > 0
        if B[i,j] = diagonal
            append v[i] to alignedV
            append w[j] to alignedW
            i <- i - 1
            j <- j - 1

        else if B[i,j] = up
            append v[i] to alignedV
            append '-' to alignedW
            i <- i - 1

        else
            append '-' to alignedV
            append w[j] to alignedW
            j <- j - 1

    reverse alignedV
    reverse alignedW

    return (alignedV,alignedW)

```

Quindi, la prima procedura costruisce la matrice degli score e quella dei predecessori; la seconda utilizza quest'ultima per ricostruire un allineamento ottimo a partire dalla cella (n, m) .

3.3 Analisi della complessità

Abbiamo allora definito l'algoritmo standard di Needleman-Wunsch. Possiamo adesso analizzarne la complessità, centro del nostro confronto.

Sia $T(n, m)$ la complessità di esecuzione, temporale, dell'algoritmo. Sia $S(n, m)$ la complessità spaziale dell'algoritmo.

Allora,

$$T(n, m) = \Theta(nm),$$

poiché la matrice contiene $(n + 1) \times (m + 1)$ stati e ciascuno viene calcolato mediante un numero costante di operazioni.

Memorizzando l'intera matrice avremo

$$S(n, m) = \Theta(nm).$$

Pertanto, per sequenze di lunghezza dello stesso ordine, tempo e spazio crescono quadraticamente rispetto alla lunghezza.

La crescita quadratica dello spazio può quindi diventare un limite per sequenze di grandi dimensioni, poiché memorizzare l'intera matrice richiede una quantità di memoria proporzionale al prodotto delle lunghezze delle due sequenze.

Inoltre, non abbiamo ancora considerato il costo del traceback. Un cammino da (n, m) a $(0, 0)$ contiene al più $n + m$ archi, quindi il traceback percorre al massimo $n + m$ celle, richiedendo tempo

$$T_{\text{traceback}}(n, m) = O(n + m).$$

Inoltre, l'allineamento prodotto può contenere al massimo $n + m$ colonne, e richiede quindi

$$O(n + m)$$

spazio per la memorizzazione dell'output.

Pertanto, il costo temporale complessivo dell'algoritmo rimane

$$T(n, m) = \Theta(nm) + O(n + m) = \Theta(nm),$$

mentre la complessità spaziale complessiva della versione standard è

$$S(n, m) = \Theta(nm) + O(n + m) = \Theta(nm).$$

Sappiamo che, per calcolare

$$S_{i,j}$$

abbiamo bisogno di

$$S_{i-1,j}, \quad S_{i,j-1}, \quad S_{i-1,j-1}.$$

Allora, possiamo notare come il calcolo della colonna j richieda soltanto la colonna $j - 1$ e i valori già calcolati della colonna j . Quindi, se riduciamo il problema soltanto al calcolo del punteggio ottimo, potremmo diminuire lo spazio necessario, andando ad usare soltanto due vettori di $n + 1$ o $m + 1$ elementi, scegliendo opportunamente la sequenza più corta, ottenendo quindi uno spazio lineare.

$$S_{\text{score}}(n, m) = \Theta(\min\{n, m\}).$$

Notiamo che la complessità nel tempo non cambia.

Tale ottimizzazione consente quindi di soddisfare il primo obiettivo computazionale, ossia il calcolo dello score ottimo, utilizzando spazio lineare. Tuttavia, eliminando le celle già utilizzate vengono perse anche le informazioni necessarie per effettuare il traceback e ricostruire un allineamento ottimo. Il problema diventa quindi quello di ottenere anche la soluzione, e non soltanto il suo valore, mantenendo una complessità spaziale lineare.

3.4 Implementazione

L'algoritmo di Needleman-Wunsch è stato implementato in C mantenendo una corrispondenza diretta tra gli oggetti introdotti nella formulazione teorica e le strutture dati utilizzate nel programma. In questa sezione vengono descritte le principali scelte implementative rilevanti ai fini del confronto sperimentale.

3.4.1 Strutture dati

Per rappresentare i simboli di una sequenza di DNA è stato definito il tipo **Base**, che fornisce una codifica numerica degli elementi di

$$\Sigma_{\text{DNA}} \cup \{-\},$$

inclusendo quindi le quattro basi azotate e un valore dedicato al gap.

Un filamento di DNA è rappresentato dal tipo **Strand**, contenente un array di tipo **Base** e la lunghezza di tale array.

La funzione associata all'algoritmo di Needleman-Wunsch restituisce un oggetto di tipo

Alignment,

formato da due sequenze, **Strand**, e da un intero, **score**, che corrisponde a $S_{n,m}$.

Poiché nella formulazione teorica le sequenze sono indicizzate a partire da 1, mentre gli array C partono da 0, al simbolo v_i corrisponde nel codice `v->bases[i-1]`, e analogamente w_j corrisponde a `w->bases[j-1]`.

Nel programma è stato utilizzato il seguente modello di scoring:

$$\text{match} = 1, \quad \text{mismatch} = -1, \quad \text{indel} = -1.$$

La funzione

`scoring(a,b,model)`

implementa la funzione di scoring teorica $\delta(a, b)$.

L'implementazione alloca due matrici, **score** e **pred**, entrambe di dimensione $(n+1) \times (m+1)$. La prima memorizza gli score $S_{i,j}$ associati agli stati della programmazione dinamica, mentre la seconda memorizza le informazioni relative ai predecessori.

$$\text{score} \leftrightarrow S, \quad \text{pred} \leftrightarrow B.$$

Entrambe le matrici sono allocate dinamicamente come blocchi contigui di memoria. Questa scelta consente di mantenere la rappresentazione bidimensionale `score[i][j]`, limitando l'overhead dovuto a molteplici allocazioni e garantendo una disposizione regolare degli elementi in memoria.

La matrice **pred** utilizza il tipo **Direction**; le possibili direzioni sono rappresentate mediante flag distinti e combinate attraverso una maschera di bit. In questo modo una singola cella può memorizzare contemporaneamente più predecessori ottimi.

3.4.2 Algoritmo

Dopo l'inizializzazione della prima riga e della prima colonna delle matrici, gli stati interni vengono calcolati secondo la ricorrenza descritta nella Sezione 3.1. Il seguente frammento mostra il riempimento delle matrici **score** e **pred**.

```
for(size_t i=1; i<=n; i++) {
    for(size_t j=1; j<=m; j++) {
        int up = score[i-1][j] + scoring(v->bases[i-1],BASE_GAP,model);
        int left = score[i][j-1] + scoring(BASE_GAP,w->bases[j-1],model);
        int diagonal = score[i-1][j-1] + scoring(v->bases[i-1],w->bases[j-1],model);
        score[i][j] = max(up,left,diagonal);

        pred[i][j] = NONE;
        if(score[i][j] == up) pred[i][j] |= UP;
        if(score[i][j] == left) pred[i][j] |= LEFT;
        if(score[i][j] == diagonal) pred[i][j] |= DIAGONAL;
    }
}
```

A differenza dello pseudocodice della sezione 3.2, la scelta del predecessore non viene effettuata durante il riempimento della matrice. In caso di parità, tutte le direzioni che realizzano il massimo vengono memorizzate in `pred[i][j]` mediante una bitmask. L'operatore OR bit a bit consente infatti di aggiungere un flag senza eliminare quelli già presenti.

La selezione di un singolo predecessore avviene soltanto durante il traceback, secondo la politica deterministica

$$\text{DIAGONAL} \succ \text{UP} \succ \text{LEFT}.$$

Poiché il traceback procede dalla cella (n, m) verso $(0, 0)$, le colonne dell'allineamento vengono generate in ordine inverso. Al termine della ricostruzione le due sequenze vengono quindi invertite mediante

```
reverse(aligned_v, length);
reverse(aligned_w, length);
```

La funzione `needleman_wunsch` restituisce un oggetto di tipo `Alignment`, dove

$$\text{alignment.score} = \text{score}[n][m] = \text{OPT}(v, w).$$

Le matrici `score` e `pred` costituiscono strutture ausiliarie necessarie soltanto durante l'esecuzione dell'algoritmo. Dopo il traceback, una volta trasferiti nella struttura `Alignment` lo score finale e le sequenze allineate, la memoria associata alle due matrici può essere liberata.

```
free(score);
free(pred);
```

L'implementazione riflette quindi direttamente la complessità spaziale analizzata nella sezione precedente: sia `score` sia `pred` richiedono un numero di celle proporzionale a $(n + 1) \times (m + 1)$. Sarà proprio l'eliminazione della necessità di conservare tali matrici complete a motivare l'algoritmo di Hirschberg.

4 Algoritmo di Hirschberg

Nel capitolo precedente abbiamo osservato che, se siamo interessati soltanto al calcolo dello score ottimo del problema dell'allineamento globale, non è necessario memorizzare l'intera matrice della programmazione dinamica. Mantenendo solamente le colonne necessarie al calcolo dello stato successivo, lo spazio ausiliario può infatti essere ridotto a

$$\Theta(\min\{n, m\}).$$

Questa ottimizzazione, tuttavia, non permette direttamente di ricostruire un allineamento ottimo, poiché le informazioni necessarie al traceback vengono progressivamente eliminate.

L'obiettivo dell'algoritmo di Hirschberg è mantenere una complessità spaziale lineare, ricostruendo comunque un allineamento associato allo score massimo.

Per ottenere questo risultato, Hirschberg combina il calcolo degli score in spazio lineare con una strategia *divide et impera* applicata all'alignment graph. Consideriamo infatti un cammino ottimo da $(0, 0)$ a (n, m) . Tale cammino deve attraversare la colonna centrale del grafo,

$$j_{\text{mid}} = \left\lfloor \frac{m}{2} \right\rfloor.$$

Se riusciamo quindi a individuare un vertice della colonna j_{mid} appartenente ad almeno un cammino ottimo, possiamo suddividere il problema in due sottoproblemi: la ricerca di un cammino ottimo dalla sorgente a tale vertice e la ricerca di un cammino ottimo dal vertice al sink.

Applicando ricorsivamente questa procedura è possibile ricostruire il cammino ottimo senza memorizzare l'intera matrice dei predecessori.

Nella formulazione utilizzata nel seguito, l'algoritmo individua non soltanto un vertice centrale, detto *middle vertex*, ma anche un arco ottimo uscente da esso, detto *middle edge*. Tale arco permette di determinare una colonna dell'allineamento e di separare precisamente i due sottoproblemi ricorsivi.

4.1 Ricorrenza, Middle vertex e Middle edge

La ricorrenza utilizzata nell'algoritmo di Hirschberg è la stessa introdotta per Needleman-Wunsch. Sia $S_{i,j}$ lo score ottimo dell'allineamento tra i prefissi $v_1 \cdots v_i$ e $w_1 \cdots w_j$. Per $i, j > 0$ vale quindi

$$S_{i,j} = \max \begin{cases} S_{i-1,j} + \delta(v_i, -), \\ S_{i,j-1} + \delta(-, w_j), \\ S_{i-1,j-1} + \delta(v_i, w_j) \end{cases}$$

Anche il modello di scoring δ rimane invariato rispetto all'implementazione di Needleman-Wunsch. In questo modo i due algoritmi risolvono lo stesso problema di ottimizzazione e possono successivamente essere confrontati utilizzando le medesime condizioni sperimentali.

La prima osservazione su cui si basa Hirschberg riguarda il calcolo degli score. Per determinare una cella $S_{i,j}$ sono necessari solamente i valori

$$S_{i-1,j}, \quad S_{i,j-1}, \quad S_{i-1,j-1}.$$

Di conseguenza, durante il calcolo della colonna j è sufficiente conservare la colonna precedente e i valori già calcolati della colonna corrente.

Senza perdita di generalità, supponiamo $n \leq m$, disponendo quindi la sequenza più corta lungo le righe dell'alignment graph. Gli score possono essere memorizzati in due vettori

$$\text{previous}, \quad \text{current},$$

entrambi contenenti $n + 1$ elementi.

Una volta terminato il calcolo di una colonna, i ruoli dei due vettori vengono scambiati: la colonna appena calcolata diventa la colonna precedente utilizzata per determinare quella successiva. Non è quindi necessario conservare le colonne già elaborate.

Consideriamo adesso la colonna centrale

$$j_{\text{mid}} = \left\lfloor \frac{m}{2} \right\rfloor.$$

Ogni cammino dalla sorgente $(0,0)$ al pozzo (n,m) attraversa almeno un vertice della colonna j_{mid} . Vogliamo determinare quale di questi vertici appartiene ad almeno un cammino ottimo.

Per ogni $i \in \{0, \dots, n\}$ definiamo

$$\text{prefix}(i) = \text{OPT}[(0,0) \rightarrow (i, j_{\text{mid}})],$$

ossia lo score del miglior cammino dalla sorgente al vertice (i, j_{mid}) .

In maniera analoga, definiamo

$$\text{suffix}(i) = \text{OPT}[(i, j_{\text{mid}}) \rightarrow (n, m)],$$

ossia lo score del miglior cammino del medesimo vertice al sink.

Definiamo quindi

$$\text{length}(i) = \text{prefix}(i) + \text{suffix}(i).$$

La quantità $\text{length}(i)$ rappresenta il massimo score ottenibile tra tutti i cammini dalla sorgente al pozzo vincolati a passare attraverso (i, j_{mid}) .

Poiché ogni cammino dalla sorgente al pozzo deve attraversare la colonna j_{mid} , lo score ottimo globale può essere espresso come

$$\text{OPT} = \max_{0 \leq i \leq n} \{\text{prefix}(i) + \text{suffix}(i)\}.$$

Allora, scegliendo

$$i_{\text{mid}} \in \arg \max_{0 \leq i \leq n} \text{length}(i),$$

il vertice $(i_{\text{mid}}, j_{\text{mid}})$ appartiene ad almeno un cammino ottimo, e viene chiamato *middle vertex*.

La decomposizione è possibile grazie alla sottostruttura ottima del problema: se il prefisso o il suffisso di un cammino ottimo, che attraversa tale vertice, non fosse a sua volta ottimo per il corrispondente sottoproblema, sarebbe possibile sostituirlo con un cammino di score maggiore e ottenere uno score complessivo superiore, in contraddizione con l'ottimalità del cammino originario.

I valori $\text{prefix}(i)$ possono essere calcolati mediante la programmazione dinamica in spazio lineare descritta precedentemente, arrestando il calcolo alla colonna j_{mid} .

Per ottenere $\text{suffix}(i)$ possiamo applicare lo stesso procedimento nel verso opposto. Concettualmente, ciò equivale a considerare l'alignment graph con gli archi invertiti e a calcolare il miglior cammino dal pozzo verso la colonna centrale.

Quindi, è sufficiente eseguire la stessa ricorrenza percorrendo le sequenze in ordine inverso, come se avessimo invertito la direzione degli archi.

Una volta determinato il middle vertex rimane da individuare quale arco uscente da esso appartenga a un cammino ottimo.

Dal middle vertex possono partire al più tre archi, corrispondenti alle tre operazioni dell'allineamento.

Sia $R(i, j)$ lo score del miglior cammino dal vertice (i, j) al pozzo. Sia e l'arco avente come coda il middle vertex, allora definiamo i tre valori candidati

$$\begin{aligned} c(e_{\downarrow}) &= \delta(v_{i+1}, -) + R(i+1, j), \\ c(e_{\rightarrow}) &= \delta(-, w_{j+1}) + R(i, j+1), \\ c(e_{\searrow}) &= \delta(v_{i+1}, w_{j+1}) + R(i+1, j+1). \end{aligned}$$

Allora, il middle edge viene scelto tra gli archi che massimizzano tali quantità.

Il termine relativo al cammino dalla sorgente al middle vertex non compare nel confronto, poiché è lo stesso per tutti gli archi uscenti da tale vertice. È quindi sufficiente confrontare il peso dell'arco con lo score ottimo del cammino che parte dal suo estremo finale.

Una volta individuato il middle edge, il problema viene suddiviso nei due sottoproblemi posti rispettivamente prima e dopo tale arco.

La ricostruzione dell'allineamento assume quindi la forma

$$A = A_{\text{left}} \circ e_{\text{mid}} \circ A_{\text{right}},$$

dove A_{left} e A_{right} sono gli allineamenti ottenuti ricorsivamente nei due sottoproblemi, mentre e_{mid} determina una singola colonna dell'allineamento complessivo.

In questo modo Hirschberg non necessita di una matrice globale dei predecessori; le colonne dell'allineamento vengono prodotte progressivamente durante la ricorsione.

Analizziamo un esempio per verificare ciò che abbiamo appena detto. Consideriamo le solite sequenze $v := \text{ACCT}$ e $w := \text{ACT}$. La colonna centrale è:

$$j_{\text{mid}} = \left\lfloor \frac{3}{2} \right\rfloor = 1$$

I calcoli degli score in avanti e in senso inverso restituiscono:

$$\text{prefix} = \begin{bmatrix} -1 \\ 1 \\ 0 \\ -1 \\ -2 \end{bmatrix}, \quad \text{suffix} = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 0 \\ -2 \end{bmatrix}$$

Allora,

$$\text{length} = \begin{bmatrix} -1 \\ 2 \\ 2 \\ -1 \\ -4 \end{bmatrix}$$

Abbiamo due possibili scelte per il middle vertex, $(1, 1)$ e $(2, 1)$. Senza perdita di generalità adottiamo

$$(i_{\text{mid}}, j_{\text{mid}}) = (1, 1)$$

Calcoliamo adesso il middle edge dal vertice $(1, 1)$

$$c(e_{\downarrow}) = \delta(C, -) + R(2, 1) = -1 + 2 = 1,$$

$$c(e_{\rightarrow}) = \delta(-, C) + R(1, 2) = -1 - 1 = -2,$$

$$c(e_{\searrow}) = \delta(C, C) + R(2, 2) = 1 + 0 = 1.$$

Anche questa volta troviamo un pareggio; adottando la politica di tie-breaking della Sezione 3.4.2, l'arco scelto è

$$(1, 1) \rightarrow (2, 2)$$

Infine, possiamo visualizzare il paradigma divide et impera nel nostro esempio:

$$(0, 0) \rightsquigarrow (1, 1) \quad \underbrace{(1, 1) \rightarrow (2, 2)}_{\text{middle edge}} \quad (2, 2) \rightsquigarrow (4, 3)$$

4.2 Pseudocodice

Possiamo unire i risultati precedenti nello pseudocodice dell'algoritmo di Hirschberg.

Sia (i, j) il vertice dell'alignment graph che rappresenta l'allineamento dei prefissi $v_1 \cdots v_i$ e $w_1 \cdots w_j$.

Un sottoproblema viene identificato da un rettangolo

$$[\text{top}, \text{bottom}] \times [\text{left}, \text{right}]$$

dell'alignment graph, avente come sorgente il vertice $(\text{top}, \text{left})$ e come pozzo $(\text{bottom}, \text{right})$.

Tramite questa rappresentazione possiamo analizzare le direzioni possibili per gli archi all'interno del rettangolo. Un arco verticale da (i, j) a $(i+1, j)$ consuma il simbolo v_{i+1} . Un arco orizzontale da (i, j) a $(i, j+1)$ consuma w_{j+1} , mentre un arco diagonale consuma entrambi.

Calcolo degli score in spazio lineare. Il primo componente necessario è una versione di Needleman-Wunsch che, dato un rettangolo dell'alignment graph, restituisce gli score dei vertici appartenenti alla sua ultima colonna.

Come osservato precedentemente, per calcolare una colonna sono sufficienti la colonna precedente e i valori già determinati dalla colonna corrente. Indichiamo tali vettori rispettivamente con **previous** e **current**.

Nello pseudocodice seguente gli indici dei vettori coincidono, per semplicità, con gli indici delle righe dell'alignment graph.

Listing 3: Calcolo degli score in spazio lineare

```
NWScore(v,w,top,bottom,left,right,delta)
  previous[top] <- 0

  for i <- top+1,...,bottom
    previous[i] <- previous[i-1] + delta(v[i],'-')

  for j <- left+1,...,right
    current[top] <- previous[top] + delta('-',w[j])

    for i <- top+1,...,bottom
      vertical <- current[i-1] + delta(v[i],'-')
      horizontal <- previous[i] + delta('-',w[j])
      diagonal <- previous[i-1] + delta(v[i],w[j])
      current[i] <- max(vertical,horizontal,diagonal)

    swap(previous,current)

  return previous
```

Al termine della procedura,

$$\text{previous}[i] = \text{OPT}[(\text{top}, \text{left}) \rightarrow (i, \text{right})],$$

per ogni $i \in \{\text{top}, \dots, \text{bottom}\}$.

Possiamo notare che, quando $\text{right} = j_{\text{mid}}$, questo vettore contiene esattamente i valori che in precedenza chiamavamo $\text{prefix}(i)$.

Per il calcolo dei valori $\text{suffix}(i)$ utilizziamo una procedura simmetrica, **NWScoreReverse**, che visita il rettangolo a partire dal pozzo.

In questo caso il vettore soddisfa

$$\text{previous}[i] = \text{OPT}[(i, \text{left}) \rightarrow (\text{bottom}, \text{right})].$$

Listing 4: Calcolo inverso degli score

```
NWScoreReverse(v,w,top,bottom,left,right,delta)
  previous[bottom] <- 0

  for i <- bottom-1,...,top
    previous[i] <- previous[i+1] + delta(v[i+1],'-')

  for j <- right-1,...,left
    current[bottom] <- previous[bottom] + delta('-',w[j+1])

    for i <- bottom-1,...,top
      vertical <- current[i+1] + delta(v[i+1],'-')
      horizontal <- previous[i] + delta('-',w[j+1])
      diagonal <- previous[i+1] + delta(v[i+1],w[j+1])
      current[i] <- max(vertical,horizontal,diagonal)

    swap(previous,current)

  return previous
```

Determinazione del middle edge. Una volta individuato il middle vertex (i, j) , rimane da scegliere l'arco ottimo uscente da esso.

Per effettuare il confronto sono necessari gli score a partire dalla colonna j e dalla colonna $j + 1$. Indichiamo con `suffix` e `suffix_next` i due vettori corrispondenti.

Listing 5: Determinazione del middle edge

```
MiddleEdge(v,w,bottom,mid_vertex,suffix,suffix_next,delta)
  i <- mid_vertex.i
  j <- mid_vertex.j

  if i = bottom
    return (i,j)->(i,j+1)

  vertical <- suffix[i+1] + delta(v[i+1],'-')
  horizontal <- suffix_next[i] + delta('-',w[j+1])
  diagonal <- suffix_next[i+1] + delta(v[i+1],w[j+1])
  best <- max(vertical,horizontal,diagonal)

  if best = diagonal
    return (i,j)->(i+1,j+1)

  else if best = vertical
    return (i,j)->(i+1,j)

  else
    return (i,j)->(i,j+1)
```

In caso di parità tra più valori candidati, è sufficiente scegliere uno qualunque degli archi che realizzano il massimo. Nel seguito adottiamo, per determinismo, la priorità diagonale, verticale, orizzontale.

Per individuare il middle edge sono necessari sia

$$R(i, j_{\text{mid}})$$

sia

$$R(i, j_{\text{mid}} + 1).$$

Entrambi possono essere ottenuti mediante `NWScoreReverse`, applicata rispettivamente alla colonna e a quella immediatamente successiva.

Procedura ricorsiva. Possiamo infine combinare le procedure precedenti.

I casi base verificano quando uno dei due lati del rettangolo ha lunghezza nulla. Se `left = right`, la sottostringa $v_{\text{top}+1} \cdots v_{\text{bottom}}$ viene allineata con la stringa vuota e l'allineamento contiene quindi soltanto archi verticali. Analogamente, se `top = bottom`, la sottostringa $w_{\text{left}+1} \cdots w_{\text{right}}$ viene allineata con la stringa vuota mediante soli archi orizzontali.

Negli altri casi viene individuato il middle edge e il problema viene risolto ricorsivamente nei due rettangoli che esso separa.

Listing 6: Algoritmo di Hirschberg

```
Hirschberg(v,w,top,bottom,left,right,delta)

  if left = right
    return allineamento verticale di v[top+1,...,bottom]

  if top = bottom
    return allineamento orizzontale di w[left+1,...,right]
```

```

middle <- floor((left + right)/2)

prefix <- NWScore(v,w,top,bottom,left,middle,delta)
suffix <- NWScoreReverse(v,w,top,bottom,middle,right,delta)

mid_i <- top

for i <- top+1,...,bottom
  if prefix[i] + suffix[i] > prefix[mid_i] + suffix[mid_i]
    mid_i <- i

mid_vertex <- (mid_i,middle)

suffix_next <- NWScoreReverse(v,w,top,bottom,middle+1,right,delta)

mid_edge <- MiddleEdge(v,w,bottom,mid_vertex,suffix,suffix_next,delta)

left_alignment <- Hirschberg(v,w,top,mid_i,left,middle,delta)
right_alignment <- Hirschberg(v,w,mid_edge.to_i,bottom,mid_edge.to_j,right,delta)

return concatenate(left_alignment,mid_edge,right_alignment)

```

Tramite un lieve abuso di notazione, nel listing la concatenazione con `mid_edge` indica l'aggiunta della singola colonna di allineamento determinata da tale arco.

4.3 Analisi della complessità

Studiamo adesso la complessità temporale e spaziale dell'algoritmo di Hirschberg.

Siano $n := |v|$ e $m := |w|$. Supponiamo, senza perdita di generalità, $n \leq m$, così da disporre la sequenza più corta lungo le righe dell'alignment graph.

Analizziamo innanzitutto il costo di un singolo sottoproblema. Come visto nella sezione precedente, esso corrisponde a un rettangolo dell'alignment graph; siano h e k rispettivamente la sua altezza e la sua larghezza.

La procedura `NWScore` visita ogni stato del rettangolo un numero costante di volte, richiedendo quindi tempo

$$\Theta(hk).$$

Essa mantiene solamente due vettori di $h + 1$ elementi, utilizzando quindi spazio

$$\Theta(h).$$

La stessa analisi vale per `NWScoreReverse`.

Per individuare il middle vertex e il middle edge viene eseguito un numero costante di scansioni del rettangolo. Sebbene ciò introduca un fattore costante aggiuntivo rispetto a una singola esecuzione di Needleman-Wunsch, il costo temporale complessivo associato a un rettangolo di dimensione $h \times k$ rimane

$$\Theta(hk).$$

Consideriamo adesso l'intera ricorsione. Dopo aver individuato il middle edge, il rettangolo corrente viene suddiviso in due sottoproblemi. Poiché la divisione avviene rispetto alla colonna centrale, l'area complessiva dei rettangoli processati diminuisce geometricamente procedendo nei livelli della ricorsione.

Sia $A_0 := nm$ l'area dell'alignment graph iniziale. Il lavoro complessivo può essere limitato, a meno di un fattore costante dovuto alle scansioni necessarie per ogni sottoproblema, dalla serie geometrica

$$A_0 + \frac{A_0}{2} + \frac{A_0}{4} + \dots = \sum_{\ell=0}^{\infty} \frac{A_0}{2^\ell} = 2A_0.$$

Ne segue che

$$T(n, m) = O(nm)$$

D'altra parte, già nella prima chiamata è necessario calcolare gli score relativi a una porzione complessivamente proporzionale all'intero alignment graph, richiedendo $\Omega(nm)$ operazioni. Allora

$$\boxed{T(n, m) = \Theta(nm)}$$

Hirschberg conserva dunque la complessità temporale asintotica rispetto a Needleman-Wunsch. Il vantaggio riguarda l'utilizzo della memoria: la ricostruzione divide et impera richiede di ricalcolare alcuni score, introducendo un maggiore fattore costante nel tempo di esecuzione, ma evita di memorizzare l'intera matrice della programmazione dinamica.

Concentriamoci quindi adesso sulla complessità spaziale.

In un sottoproblema di altezza h , i vettori utilizzati dalle procedure **NWScore** e **NWScoreReverse** contengono $h + 1$ elementi. Poiché durante la determinazione del middle edge viene mantenuto un numero costante di tali vettori, la memoria richiesta dalle strutture di programmazione dinamica è

$$S_{\text{DP}}(n, m) = \Theta(\min\{n, m\}).$$

I vettori utilizzati per individuare il middle edge sono necessari soltanto all'interno della chiamata corrente e possono essere liberati prima di procedere ricorsivamente con i due sottoproblemi. Conseguentemente, la memoria occupata dai vettori dei diversi livelli della ricorsione non si accumula quindi lungo lo stack.

Poiché a ogni livello la larghezza del sottoproblema viene circa dimezzata, la profondità della ricorsione è

$$O(\log m).$$

Ogni record di attivazione contiene soltanto un numero costante di indici e riferimenti, per cui lo stack ricorsivo richiede

$$S_{\text{stack}}(n, m) = O(\log m).$$

Lo spazio ausiliario complessivo è pertanto

$$S_{\text{aux}}(n, m) = O(\min\{n, m\} + \log \max\{n, m\}).$$

In particolare, le strutture utilizzate dalla programmazione dinamica richiedono spazio lineare nella lunghezza della sequenza più corta. È in questo senso che Hirschberg viene definito un algoritmo di allineamento in spazio lineare.

Infine consideriamo la memoria utilizzata per salvare l'output. Come osservato per Needleman-Wunsch, se L indica il numero di colonne dell'allineamento finale, vale

$$\max\{n, m\} \leq L \leq n + m.$$

Poiché

$$\max\{n, m\} \geq \frac{n + m}{2},$$

segue che

$$L = \Theta(n + m).$$

La rappresentazione esplicita delle due sequenze allineate richiede quindi

$$S_{\text{out}}(n, m) = \Theta(n + m).$$

Se viene considerato anche l'output, lo spazio complessivo è pertanto

$$S_{\text{tot}}(n, m) = \Theta(n + m).$$

Abbiamo allora visto come Hirschberg conservi la stessa complessità temporale asintotica di Needleman-Wunsch, ma elimina la necessità di memorizzare le matrici complete degli score e dei predecessori. Lo spazio richiesto dalle strutture utilizzate durante il calcolo passa da

$$\Theta(nm)$$

a uno spazio lineare nella dimensione delle sequenze, e in particolare a

$$\Theta(\min\{n, m\})$$

per i vettori della programmazione dinamica.

4.4 Implementazione

L'implementazione di Hirschberg utilizza le stesse strutture dati e lo stesso modello di scoring introdotti per l'implementazione di Needleman-Wunsch. Quindi le sequenze vengono rappresentate dal tipo **Strand** mentre l'output viene restituito attraverso una struttura **Alignment**. Mancano ovviamente le matrici **score** e **pred**.

4.4.1 Calcolo degli score in spazio lineare

I sottoproblemi ricorsivi non vengono rappresentati creando nuove sottostringhe. Ciascuno di essi è identificato tramite quattro indici,

$$v_source, \quad v_sink, \quad w_source, \quad w_sink,$$

che delimitano le porzioni delle due sequenze coinvolte, ricordando la rappresentazione degli indici del rettangolo per i sottoproblemi visti nelle sezioni precedenti.

La funzione **nw_score** implementa la procedura **NWScore** descritta nella Sezione 4.2. Quindi alloca due vettori di $n + 1$ interi, con $n = v_sink - v_source$.

Di seguito viene mostrato il frammento di codice che comprende l'inizializzazione dei due vettori e il calcolo degli score dell'ultima colonna del sottoproblema corrente.

```
previous[0] = 0;
for(size_t i=1; i<=n; i++) {
    previous[i] = previous[i-1] + scoring(v->bases[v_source + i - 1],BASE_GAP,model);
}
current[0] = 0;

for(size_t j=1; j<=m; j++) {
    current[0] = previous[0] + scoring(BASE_GAP,w->bases[w_source + j - 1],model);
    for(size_t i=1; i<=n; i++) {
        int up      = current[i-1] + scoring(v->bases[v_source + i - 1],BASE_GAP,model);
        int left     = previous[i] + scoring(BASE_GAP,w->bases[w_source + j - 1],model);
        int diagonal = previous[i-1] + scoring(v->bases[v_source + i - 1],w->bases[w_source + j - 1],model);
        current[i] = max(up,left,diagonal);
    }
    int *temp = previous;
```

```

    previous = current;
    current  = temp;
}

```

Gli indici *i* e *j* utilizzati all'interno della funzione sono relativi al sottoproblema corrente. Per accedere alla sequenza originale vengono quindi sommati gli offset *v_source* e *w_source*.

Terminato il calcolo, il vettore non più necessario viene liberato e *previous* viene restituito al chiamante.

La funzione *nw_score_reverse* realizza invece il calcolo simmetrico necessario per gli score di suffisso. In questo caso l'inizializzazione parte dall'ultima posizione del vettore e le sequenze vengono percorse in ordine decrescente, di seguito è possibile vederne il codice.

```

previous[n] = 0;
for(size_t i = n; i --> 0; ) {
    previous[i] = previous[i+1] + scoring(v->bases[v_source + i], BASE_GAP, model);
}

```

È interessante anche vedere come è stata affrontata l'indicizzazione dei cicli nel verso contrario:

```

for(size_t j = m; j --> 0; ) {
    [...]
    for(size_t i = n; i --> 0; ) {
        [...]
    }
}

```

4.4.2 Middle vertex e middle edge

Per la ricerca del middle vertex vengono chiamate le funzioni *nw_score* e *nw_score_reverse*, i cui risultati vengono salvati all'interno di due vettori, *prefix* e *suffix*.

```

size_t mid_j = w_source + m/2;

int *prefix = nw_score(v,v_source,v_sink,w,w_source,mid_j,model);
int *suffix = nw_score_reverse(v,v_source,v_sink,w,mid_j, w_sink,model);

```

Il seguente frammento mostra come viene ricercato l'elemento massimo di *prefix(i) + suffix(i)*

```

size_t mid_offset = 0;
int max = prefix[0] + suffix[0];
for(size_t i=1; i<=n; i++) {
    int score = prefix[i] + suffix[i];
    //length[i] = prefix[i] + suffix[i]
    if(max < score) {
        max = score;
        mid_offset = i;
    }
}
size_t mid_i = v_source + mid_offset;

```

Poiché i vettori sono indicizzati relativamente al sottoproblema, *mid_offset* identifica la riga del middle vertex rispetto a *v_source*.

Successivamente viene calcolato il vettore *suffix_next* tramite una chiamata a *nw_score_reverse*, alla quale viene passato l'indice *mid_j + 1*.

Il middle edge viene rappresentato da una struttura `MiddleEdge`, che memorizza il vertice iniziale, il vertice finale e la direzione dell'arco.

Di seguito è presente la funzione `mid_edge` che restituisce il middle edge di un sottoproblema.

```

MiddleEdge mid_edge(size_t top, size_t bottom,
                    size_t mid_vertex_i, size_t mid_vertex_j,
                    const int *suffix, const int* suffix_next,
                    const Strand *v, const Strand *w, const scoringModel *model) {

    size_t i = mid_vertex_i - top;

    if(mid_vertex_i == bottom) {
        MiddleEdge mid_edge = {
            .from_i = mid_vertex_i,
            .from_j = mid_vertex_j,
            .to_i   = mid_vertex_i,
            .to_j   = mid_vertex_j + 1,
            .edge   = EDGE_HORIZONTAL
        };
        return mid_edge;
    }

    int next_down    = suffix[i + 1]      + scoring(v->bases[mid_vertex_i],BASE_GAP,model);
    int next_right   = suffix_next[i]      + scoring(BASE_GAP,w->bases[mid_vertex_j],model);
    int next_diagonal = suffix_next[i + 1] + scoring(v->bases[mid_vertex_i],w->bases[mid_vertex_j],
    model);
    int final = max(next_down,next_right,next_diagonal);

    MiddleEdge mid_edge = {
        .from_i = mid_vertex_i,
        .from_j = mid_vertex_j,
    };

    if(final == next_diagonal) {
        mid_edge.to_i = mid_vertex_i + 1;
        mid_edge.to_j = mid_vertex_j + 1;
        mid_edge.edge = EDGE_DIAGONAL;
    } else if(final == next_down) {
        mid_edge.to_i = mid_vertex_i + 1;
        mid_edge.to_j = mid_vertex_j;
        mid_edge.edge = EDGE_VERTICAL;
    } else {
        mid_edge.to_i = mid_vertex_i;
        mid_edge.to_j = mid_vertex_j + 1;
        mid_edge.edge = EDGE_HORIZONTAL;
    }

    return mid_edge;
}

```

Nella formulazione teorica l'arco uscente dal vertice (i, j) utilizza v_{i+1} e w_{j+1} . Gli array `C` sono indicizzati a partire da zero; dunque, se `mid_vertex_i` e `mid_vertex_j` rappresentano direttamente le coordinate del vertice, i simboli consumati dall'arco sono memorizzati in `v->bases[mid_vertex_i]` e `w->bases[mid_vertex_j]`.

4.4.3 Ricostruzione ricorsiva dell'allineamento

Avendo trovato middle vertex e middle edge siamo pronti per poter ricostruire l'allineamento.

Prima di analizzare la procedura ricorsiva citiamo velocemente i casi base, che sono stati implementati in maniera fedele al Listing 6; se una delle due sequenze è vuota, l'unico allineamento possibile consiste nell'allineare tutti i simboli dell'altra sequenza con gap.

Vediamo adesso la ricorrenza che concatena l'allineamento sinistro, il middle edge e l'allineamento destro.

```
free(prefix);
free(suffix);
free(suffix_next);

hirschberg_rec(v, v_source, mid_i, w, w_source, mid_j, model, alignment, length); // sinistra
emit(alignment, length, &edge, v, w, model);
hirschberg_rec(v, edge.to_i, v_sink, w, edge.to_j, w_sink, model, alignment, length); // destra
```

La ricostruzione dell'allineamento segue direttamente la decomposizione

$$A = A_{\text{left}} \circ e_{\text{mid}} \circ A_{\text{right}}$$

Prima di iniziare la ricorsione, la funzione pubblica `hirschberg` dispone lungo le righe la sequenza più corta. Se necessario, i due input vengono temporaneamente scambiati. In questo modo i vettori utilizzati dalle procedure di programmazione dinamica hanno sempre dimensione

$$\min\{n, m\} + 1,$$

realizzando concretamente il limite spaziale discusso nella Sezione 4.3.

5 Metodologia sperimentale

Dopo aver analizzato teoricamente e implementato gli algoritmi di Needleman-Wunsch e Hirschberg, possiamo procedere al loro confronto sperimentale.

L'obiettivo dell'analisi è verificare empiricamente le proprietà discusse nei capitoli precedenti, concentrandoci sul tempo di esecuzione e sul consumo di memoria al crescere della lunghezza delle sequenze.

Entrambi gli algoritmi sono stati eseguiti sulle medesime coppie di sequenze, usando lo stesso modello di scoring, così da permettere un confronto adeguato.

Nel seguito vengono descritti il dataset utilizzato, l'ambiente sul quale si è lavorato e il protocollo adottato per la raccolta delle misure, in modo tale da poter valutare la validità dell'esperimento, che verrà presentato nel capitolo successivo.

5.1 Dataset

Per il confronto sperimentale sono state utilizzate sequenze biologiche rappresentate nel formato FASTA.

Per ogni esperimento vengono considerate due sequenze v e w , di lunghezza rispettivamente n e m .

Poiché la complessità dei due algoritmi dipende dal prodotto $n \times m$, è necessario considerare input di dimensione crescente, in modo tale da osservare l'andamento del tempo di esecuzione e della memoria richiesta al crescere della dimensione del problema.

Per studiare la scalabilità degli algoritmi, dalle sequenze originali vengono estratti prefissi di lunghezza crescente. in modo tale da controllare la dimensione dell'input mantenendo il contenuto proveniente da sequenze biologiche reali.

5.2 Ambiente hardware e software

Tutti gli esperimenti sono stati eseguiti sul medesimo sistema, in modo da mantenere costanti le principali condizioni che possono influenzare le prestazioni.

Componente	Configurazione
Processore	...
Architettura	ARM64
Memoria RAM	...
Sistema operativo host	...
Ambiente di esecuzione	WSL2 / Ubuntu ...
Compilatore	...
Versione compilatore	...
Flag di compilazione	...

Tabella 1: Ambiente utilizzato per gli esperimenti.

5.3 Misura dei tempi e della memoria

La misura del tempo riguarda esclusivamente l'esecuzione dell'algoritmo di allineamento. Le operazioni di lettura e parsing dei file FASTA vengono quindi effettuate prima dell'inizio della misurazione, in modo da non includere nel confronto il costo dell'I/O.

Per ogni coppia di sequenze sono state fatte più esecuzioni, in modo tale da poter rendere l'esperimento indipendente dalla variabilità.

Per il consumo di memoria viene misurato il massimo utilizzo di memoria raggiunto dal processo durante l'esecuzione.

Tale misura comprende non soltanto le strutture ausiliarie proprie dell'algoritmo, ma anche l'input, l'output e l'overhead associato al processo. Poiché tuttavia entrambi gli algoritmi vengono eseguiti nelle medesime condizioni e sulle stesse sequenze, essa consente comunque un confronto empirico della diversa crescita della memoria richiesta.

Prima di utilizzare le misure prestazionali, viene inoltre verificata la correttezza del risultato confrontando gli score restituiti dai due algoritmi. Poiché il problema può ammettere più allineamenti ottimi, non è richiesto che le due sequenze allineate coincidano esattamente; è invece necessario che entrambi gli algoritmi producano un allineamento valido avente lo stesso score ottimo.

6 Risultati e confronto

6.1 Tempo di esecuzione

6.2 Consumo di memoria

6.3 Scalabilità

6.4 Needleman-Wunsch vs Hirschberg